

AI Assisted Coding

Lab Assignment 13.4

K.SUNNY

2303A51864

B:13

Task 1:RefactoringDataTransformationLogic

TaskDescription:

- Reviewthelegacycodethatcomputestransformedvaluesusing an explicit loop.
- UseanAltooltosuggestamorePythonicrefactoringapproach.
- Refactorthecodeusinglistcomprehensionsorhelperfunctions while preserving the output.

PROMPT:

RefactorthefollowingPythoncodetomakeitshorterandmorePythonicusinglist comprehension while keeping the same output.

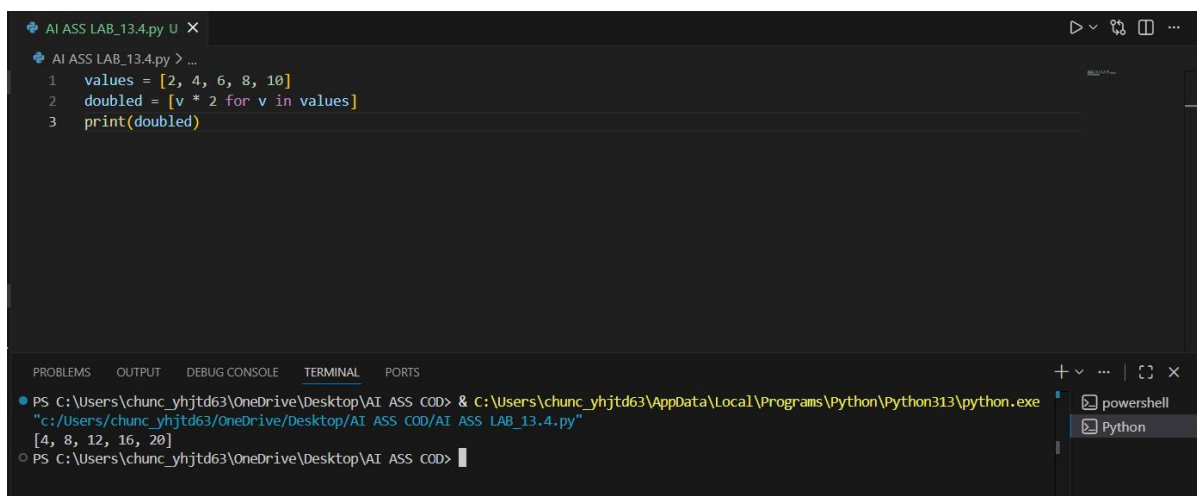
```
values=[2,4,6,8,10] doubled =
```

```
[]
```

```
forvinvalues:
```

```
    doubled.append(v*2)
```

```
print(doubled)
```



The screenshot shows a code editor with a file named 'AI ASS LAB_13.4.py'. The code is as follows:

```
1 values = [2, 4, 6, 8, 10]
2 doubled = [v * 2 for v in values]
3 print(doubled)
```

The bottom panel of the editor shows the terminal output:

```
PS C:\Users\chunc_yhjtd63\OneDrive\Desktop\AI ASS COD> & C:\Users\chunc_yhjtd63\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/chunc_yhjtd63/OneDrive/Desktop/AI ASS COD/AI ASS LAB_13.4.py"
[4, 8, 12, 16, 20]
PS C:\Users\chunc_yhjtd63\OneDrive\Desktop\AI ASS COD>
```

Task 2:ImprovingTextProcessingCodeReadability

- Analyze the legacy code that constructs a sentence using repeated string concatenation.
- Ask Alto suggest a more efficient and readable approach.

Refactor the code accordingly
while keeping the final output
unchanged.

PROMPT:

Refactor the following Python code to improve readability and efficiency by avoiding repeated string concatenation. Keep the same output.

```
AI ASS LAB_13.4.py U X
AI ASS LAB_13.4.py > ...
1
2
3 words = ["Refactoring", "with", "AI", "improves", "quality"]
4 message = " ".join(words)
5 print(message)
```

OUTPUT:

```
PS C:\Users\chunc_yhjtd63\OneDrive\Desktop\AI ASS COD> & C:\Users\chunc_yhjtd63\AppData\Local\Programs\Python\Python313\python.exe
"c:/Users/chunc_yhjtd63/OneDrive/Desktop/AI ASS COD/AI ASS LAB_13.4.py"
Refactoring with AI improves quality
PS C:\Users\chunc_yhjtd63\OneDrive\Desktop\AI ASS COD> |
```

Task3:Safer Access to Configuration Data

- Review the legacy code that manually checks for dictionary keys.
- Use `dict.get()` to refactor the code using a safer dictionary access method.
- Ensure the behavior remains the same for missing keys.

PROMPT:

Refactor the following Python code to use a safer dictionary access method instead of manually checking keys. Keep the same output if the key is missing.

`config={"host":"localhost","port":8080}` if

"timeout" in config:

```
print(config["timeout"])
```

else:

```
print("Default timeout used")
```

```
AI ASS LAB_13.4.py U X
AI ASS LAB_13.4.py > ...
1 config = {"host": "localhost", "port": 8080}
2 print(config.get("timeout", "Default timeout used"))
```

OUTPUT:

```
PS C:\Users\chunc_yhjtd63\OneDrive\Desktop\AI ASS COD> & C:\Users\chunc_yhjtd63\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/chunc_yhjtd63/OneDrive/Desktop/AI ASS COD/AI ASS LAB_13.4.py"
Default timeout used
```

Task4:Refactoring Conditional Logic for Scalability

Examinethemultipleif–elifconditionsusedtodetermine operations.

AskAltosuggestacleaner,scalablealternative.

Refactorthelogicusingmappingtechniqueswhilepreserving functionality.

PROMPT:

RefactorthefollowingPythoncodetoreplacemultipleif–elifconditionswithacleanerandscalable mapping approach while keeping the same output.

```
AI ASS LAB_13.4.py U
AI ASS LAB_13.4.py > action
1 action = "divide"
2 x, y = 10, 2
3
4 operations = {
5     "add": lambda a, b: a + b,
6     "subtract": lambda a, b: a - b,
7     "multiply": lambda a, b: a * b,
8     "divide": lambda a, b: a / b,
9 }
10
11 result = operations.get(action, lambda a, b: None)(x, y)
12 print(result)
```

OUTPUT:

```
PS C:\Users\chunc_yhjtd63\OneDrive\Desktop\AI ASS COD> & C:\Users\chunc_yhjtd63\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/chunc_yhjtd63/OneDrive/Desktop/AI ASS COD/AI ASS LAB_13.4.py"
5.0
PS C:\Users\chunc_yhjtd63\OneDrive\Desktop\AI ASS COD>
```

Task5:SimplifyingSearchLogicinCollections

Identifytheexplicitloopusedforsearchinganitem.

UseAllassistancetorefactorthelogicintoamoreconciseand readable form.

Maintainthesameoutputbehavior.

PROMPT:

RefactorthefollowingPythoncodetosimplifythesearchlogicinacollectionusingamoreconcisePython approach while keeping the same output.

```
AI ASS LAB_13.4.py U X
AI ASS LAB_13.4.py > inventory
1 inventory = ["pen", "notebook", "eraser", "marker"]
2 found = "eraser" in inventory
3 print("Item Available" if found else "Item Not Available")
```

OUTPUT:

```
PS C:\Users\chunc_yhjtd63\OneDrive\Desktop\AI ASS COD> & C:\Users\chunc_yhjtd63\AppData\Local\Programs\Python\Python313\python.exe
"c:/Users/chunc_yhjtd63/OneDrive/Desktop/AI ASS COD/AI ASS LAB_13.4.py"
Item Available
PS C:\Users\chunc_yhjtd63\OneDrive\Desktop\AI ASS COD> |
```